Ders 9 Genişletilmiş Notu: Uygulama-3: Struct ile Çalışan Veritabanı, Dinamik Dizi ve Bağlı Liste Karşılaştırması, Başlık Dosyaları ve Makrolar

Ders 9 Genişletilmiş Notu: Uygulama-3: Struct ile Çalışan Veritabanı, Dinamik Dizi ve Bağlı Liste Karşılaştırması, Başlık Dosyaları ve Makrolar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Uygulamanın Konusu ve Kaydın Durumu
- 2. Struct Yapılarının Tanımlanması
- 3. main Fonksiyonu ve Menü
- 4. Yeni Kişi Oluşturma: initNewPerson
- 5. Veritabanına Ekleme: AddPerson
- 6. Yazdırma ve Listeleme
- 7. İsme Göre Arama: SearchDatabase
- 8. Kayıt Silme: FreePerson ve DeleteDatabase
- 9. Bütün Belleği Serbest Bırakma: FreeDatabase
- 10. Bağlı Liste Yapısına Geçiş
- 11. Listenin Başına Ekleme: AddPersonFirst
- 12. Listeyi Dolaşma ve İlk Deneme
- 13. Listenin Sonuna Ekleme: AddPersonLast
- 14. Listede Arama
- 15. Listeden Silme: DeleteDatabase
- 16. Listeyi Serbest Bırakma ve main
- 17. Dinamik Dizi ve Bağlı Liste Karşılaştırması
- 18. Başlık Dosyaları ve Makrolar

1. Uygulamanın Konusu ve Kaydın Durumu

Bu kayıt, Yapısal Programlama dersinin üçüncü uygulamasıdır. Ahmet Hoca derste yetkiyi uygulamayı yapacak kişiye verip ayrıldı. Bu notta “hoca” sözcüğü, uygulamayı yürüten kişiyi anlatır. Kısa nota göre bu kişi bir araştırma görevlisidir.

Bu haftanın konusu struct ve bağlı liste (linked list) yapılarıdır. Amaç, bir şirkette çalışanların bilgilerini saklayan bir veritabanı gerçeklemektir. Veritabanı iki farklı şekilde yazılır: önce dizi (array) tabanlı, sonra bağlı liste tabanlı. Sonunda iki yaklaşım karşılaştırılır. Ek konu olarak başlık dosyaları ve makrolar da gösterilir.

Program bir menü üzerinden çalışır. Çalışan ekleme, listeleme, isme göre arama ve silme yapılır. Çıkarken de ayrılan bütün dinamik bellek serbest bırakılır.

Kaydın sesi çok düşüktür ve altyazının önemli bir bölümü anlamsızdır. Kod yazılırken konuşulan yerlerde altyazıdan yalnızca parçalar okunabiliyor. Bu nedenle bu nottaki kodlar, altyazıdaki anlatımla kısa notun ekran paylaşımından okuduğu kod birleştirilerek kurulmuştur. Kesin okunamayan yerler işaretlenmiştir.

2. Struct Yapılarının Tanımlanması

2.1 JOB ve PERSON

Programda iki temel struct vardır. `job` struct'ı çalışanın meslek adını, `person` struct'ı çalışanın adını, yaşını ve mesleğini tutar.

`job` içinde yalnızca mesleğin adı olacağı için tek alan yeterlidir: `char` işaretçisi cinsinden `Name`. Hoca, `struct job` yazmak yerine başına `typedef` koyup sonda kısa bir ad vererek kısaltma yaptı. Böylece her yerde `struct job` yazmak gerekmez, `JOB` yazmak aynı şeyi ifade eder.

`person` için de aynısı yapılır. Çalışanın adı `char` işaretçisi, yaşı `int` olarak tutulur. Ayrıca kişinin mesleğini gösteren bir `JOB` işaretçisi vardır. Yukarıda `JOB` kısaltması tanımlandığı için burada `struct job` yerine doğrudan `JOB` yazılabilir. Böylece bir kişi, kendi mesleğini gösteren bir alana sahip olur.

```
typedef struct job {
    char *Name;
} JOB;

typedef struct person {
    char *Name;
    int Age;
    JOB *job;
} PERSON;
```

2.2 DATABASE

Veritabanı için de bir struct ve kısaltması tanımlanır. Veritabanında `PERSON` tipinden bir dizi tutulacaktır. Hoca bunu şöyle anlattı: tek bir kişi tutulsaydı `PERSON` tipinden tek bir işaretçi yeterdi. Ama burada dizi istendiği için iki yıldız kullanılır ve elde işaretçilerden oluşan bir dizi olur.

Bu diziye ek olarak iki sayaç tutulur. `peopleCount` o anda veritabanında kaç kişi olduğunu, `databaseCount` dizinin boyutunu gösterir.

```
typedef struct database {
    PERSON **Data;
    int peopleCount;
    int databaseCount;
} DATABASE;
```

3. main Fonksiyonu ve Menü

`main` içinde önce bir `DATABASE` değişkeni tanımlanır. Başlangıçta kimse kayıtlı olmadığı için `peopleCount` sıfırdır. `databaseCount` ise ilk başta bir kişi alacak şekilde 1 yapılır. Ardından `malloc` ile `PERSON *` boyutunda, `databaseCount` eleman kadar yer ayrılır.

Menü `switch-case` yapısıyla kurulur. Her işlemden sonra menünün yeniden açılabilmesi için `switch` bir `while` döngüsünün içine alınır. Döngünün koşulu `exitControl` adlı bir değişkendir. Kullanıcının seçimi `int` tipindeki `choice` değişkenine okunur ve `switch` bu değere göre dallanır.

Seçenekler kısa nota göre şöyledir:

- `1`: kişi ekleme.
- `2`: listeleme.
- `3`: arama.
- `4`: silme.
- `0`: çıkış. Burada `exitControl` sıfır yapılır. Döngü koşulu yanlış olduğu için `while` bir daha çalışmaz ve program sonlanır.
- `default`: hiçbir seçenek tutmazsa buraya gelinir ve yanlış seçim yapıldığı yazılır.

Ekleme seçeneğinde kullanıcıdan önce isim, yaş ve meslek adı alınır. Bunlar sonra ilgili struct'lara aktarılır. Bu işi yapan fonksiyonlar sonraki bölümlerde yazılacaktır.

Ders sonrası not: Aşağıdaki `main`, altyazı ve kısa nottan birleştirilmiştir. Menü metni, `scanf` satırları ve `case 3` içindeki `-1` kontrolü derstekinden büyük olasılıkla farklıdır. Bu bölümdeki kod, sonraki bölümlerdeki fonksiyonlarla tek dosyada `gcc -Wall` ile uyarısız derlendi ve AddressSanitizer ile çalıştırıldı; ekleme, listeleme, arama, silme ve çıkış adımlarında bellek hatası çıkmadı.

Ders sonrası not: Kısa nota göre ekrandaki ilk sürümde büyük olasılıkla `int Name[100], jobName[100];` yazıyordu ve derleyici "incompatible pointer type" uyarısı verdi. İsimler için `char` dizisi gerekir, bağlı liste sürümünde bu düzeltilmiştir. Bu satır gcc 15.2 ile `gcc -Wall` altında denendi: uyarı değil, aynı adlı hatayla derleme durdu.

```c
int main(void) {
    DATABASE personDatabase;
    int exitControl = 1, choice, Age;
    char Name[100], jobName[100];

    personDatabase.peopleCount = 0;
    personDatabase.databaseCount = 1;
    personDatabase.Data = (PERSON **) malloc(sizeof(PERSON *)
        * personDatabase.databaseCount);

    while (exitControl) {
        printf("1-Add 2-List 3-Search 4-Delete 0-Exit\n");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            printf("Name, Age, JobName: ");
            scanf("%99s %d %99s", Name, &Age, jobName);
            AddPerson(&personDatabase, initNewPerson(Name, Age, jobName));
            break;
        case 2:
            ListDatabase(&personDatabase);
            break;
        case 3:
            printf("Name: ");
            scanf("%99s", Name);
            if (SearchDatabase(&personDatabase, Name) == -1) {
                printf("Person Not Found!\n");
            }
            break;
        case 4:
            printf("Name: ");
            scanf("%99s", Name);
            DeleteDatabase(&personDatabase, Name);
            break;
        case 0:
            FreeDatabase(&personDatabase);
            exitControl = 0;
            break;
        default:
            printf("Wrong choice!\n");
        }
    }
    return 0;
}
```

4. Yeni Kişi Oluşturma: initNewPerson

Alınan bilgileri struct'lara aktarmak için bir fonksiyon yazılır. Fonksiyon isim, yaş ve meslek adını parametre olarak alır ve oluşturduğu kişinin adresini döndürür.

Önce bir `PERSON` ve bir `JOB` için `malloc` ile yer ayrılır. Bunlar işaretçi olduğu için elemanlarına `->` operatörüyle erişilir.

Yaş doğrudan atanabilir, çünkü `Age` bir `int` alanıdır. İsim ise böyle atanamaz. Struct içinde ismi yalnızca işaretçi olarak tuttuk. Bu yüzden string için ayrıca yer ayırmak gerekir. Gerekli boyut, gelen ismin uzunluğudur ve `strlen` fonksiyonuyla bulunur. Ancak `strlen` yalnızca karakter sayısını verir. String'in sonuna `'\0'` karakterinin de konması gerektiği için bir fazlası alınır. Ayrılan alan `char` boyutuyla çarpılır. Yer ayrıldıktan sonra isim `strcpy` ile kopyalanır. Aynı işlem meslek adı için de yapılır.

En sonda kişi ile meslek birbirine bağlanır: yeni kişinin `job` alanına yeni mesleğin adresi atanır. Fonksiyon da yeni kişinin adresini döndürür.

```
PERSON *initNewPerson(char *Name, int Age, char *jobName) {
    PERSON *newPerson = (PERSON *) malloc(sizeof(PERSON));
    JOB *newJob = (JOB *) malloc(sizeof(JOB));

    newPerson->Name = (char *) malloc((strlen(Name) + 1) * sizeof(char));
    strcpy(newPerson->Name, Name);
    newPerson->Age = Age;

    newJob->Name = (char *) malloc((strlen(jobName) + 1) * sizeof(char));
    strcpy(newJob->Name, jobName);

    newPerson->job = newJob;
    return newPerson;
}
```

> **Hocanın vurgusu:** Struct içindeki string alanı yalnızca bir işaretçidir. String için `strlen + 1` kadar ayrı yer ayrılır ve `strcpy` ile kopyalanır.

5. Veritabanına Ekleme: AddPerson

Oluşturulan kişinin veritabanına eklenmesi gerekir. Bunun için `AddPerson` yazılır. Fonksiyona veritabanı ve yeni kişi verilir. Veritabanı adresiyle gönderilir, çünkü içindeki alanlar fonksiyon içinde değişecektir.

Yeni kişi dizinin hangi gözüne yazılacak? Dizinin ilk boş gözü `peopleCount` indisindedir. Gelen kişinin adresi bu göze atılır, sonra `peopleCount` bir artırılır. Bu ikisi tek satırda da yazılabilir: `peopleCount++` önce değeri kullanır, işlemden sonra artırır.

Sonra dizinin dolup dolmadığına bakılır. `databaseCount`, `peopleCount` değerine eşitse dizi dolmuştur ve boyutunun iki katına çıkarılması gerekir. Önce `databaseCount` iki katına çıkarılır. Bu iş `*=` operatörüyle yapılır: değişken ikiyle çarpılıp kendisine atanır.

Ardından dizi `realloc` ile büyütülür. Hoca `realloc` için şunu anlattı: bulunduğu adresten sonra yer varsa alanı olduğu yerde genişletir. Yer yoksa veriyi yeni bir yere kopyalar ve o yerin adresini döndürür. Bu nedenle `realloc` çağrısına büyütmek istediğimiz alanın adresi verilir ve dönen değer yeniden `Data`'ya atanır.

```
void AddPerson(DATABASE *personDatabase, PERSON *newPerson) {
    personDatabase->Data[personDatabase->peopleCount++] = newPerson;

    if (personDatabase->databaseCount == personDatabase->peopleCount) {
        personDatabase->databaseCount *= 2;
        personDatabase->Data = (PERSON **) realloc(personDatabase->Data,
            personDatabase->databaseCount * sizeof(PERSON *));
    }
}
```

Ders sonrası not: Kısa nota göre `realloc` sonucu doğrudan `Data`'ya atanıyor ve `NULL` kontrolü yapılmıyor. `realloc` başarısız olursa eski adres büyük olasılıkla kaybedilir. Kısa not, Ders 6'daki gibi önce geçici bir işaretçiye alıp kontrol etmeyi öneriyor.

6. Yazdırma ve Listeleme

Eklemeyi denemek için listeleme gerekir. Önce tek bir kişiyi yazdıran `PrintPerson` yazılır. Mesleğin adına ulaşmak için iki kez `->` kullanılır: `person->job->Name`. Sonra `ListDatabase` bütün kayıtları dolaşıp her biri için `PrintPerson` çağırır.

```
void PrintPerson(PERSON *person) {
    printf("Name: %s, Age: %d, JobName: %s\n",
           person->Name, person->Age, person->job->Name);
}

void ListDatabase(DATABASE *personDatabase) {
    int i;
    for (i = 0; i < personDatabase->peopleCount; i++) {
        PrintPerson(personDatabase->Data[i]);
    }
}
```

7. İsme Göre Arama: SearchDatabase

Sıradaki işlem isme göre aramadır. Fonksiyona veritabanı ve aranan isim verilir. Bütün kayıtlara bakılır. Karşılaştırma için string karşılaştırma fonksiyonu `strcmp` kullanılır: o anki kişinin `Name` alanı ile fonksiyona verilen isim karşılaştırılır. `strcmp` iki string eşitse 0 döndürür, dolayısıyla sonuç 0 ise kişi bulunmuştur.

Bulunduğunda "Person Found!" yazılır ve dönüş değeri olarak kişinin indisi verilir. Hiçbir şey bulunamazsa döngü biter ve `-1` döndürülür. Geçerli bir indis eksi bir olamayacağı için bu değer "bulunamadı" anlamına gelir.

```
int SearchDatabase(DATABASE *personDatabase, char *Name) {
    int i;
    for (i = 0; i < personDatabase->peopleCount; i++) {
        if (strcmp(personDatabase->Data[i]->Name, Name) == 0) {
            printf("Person Found!\n");
            return i;
        }
    }
    return -1;
}
```

Ders sonrası not: Kısa nota göre fonksiyon ilk yazıldığında büyük olasılıkla `void` tanımlanmış ama `return i;` içeriyordu, sonra `int` yapıldı. Altyazıda da indisin döndürüleceği söyleniyor. İndis döndüren fonksiyonun tipi `int` olmalıdır.

8. Kayıt Silme: FreePerson ve DeleteDatabase

Silme işlemi de isme göre yapılır. Yine bir isim alınır. Bu kez aramayı baştan yazmaya gerek yoktur, çünkü arama fonksiyonu zaten yazıldı. `SearchDatabase` çağrılır ve kişinin dizideki konumu öğrenilir. Sonuç `-1` ise kişi bulunamamıştır ve yapılacak bir şey yoktur.

8.1 FreePerson

Kişi varsa, o işaretçinin gösterdiği alanlar serbest bırakılır. Bu iş ayrı bir fonksiyonda toplanır, çünkü `malloc` ile oluşturulan her şeyin tek tek serbest bırakılması gerekir. Sıra şöyledir:

1. Kişinin adı (`person->Name`).
2. Mesleğin adı (`person->job->Name`).
3. Mesleğin kendisi (`person->job`).

4. Kişinin kendisi (`person`).

```
void FreePerson(PERSON *person) {
    free(person->Name);
    free(person->job->Name);
    free(person->job);
    free(person);
}
```

Hocanın vurgusu: Serbest bırakma sırası önce iç alanlar, sonra struct'ın kendisidir.

8.2 Diziyi Kaydırma

Kişi silindikten sonra dizinin o bölgesinde geçersiz bir adres kalır. Listeleme sırasında bu gözde gösterilecek geçerli bir kayıt olmaz. Bu yüzden dizi kaydırılır: silinen konumdan itibaren her göze kendisinden sonraki gözün değeri taşınır. Böylece boşluk ortadan kalkar. Son olarak bir kişi eksildiği için `peopleCount` bir azaltılır.

```
void DeleteDatabase(DATABASE *personDatabase, char *Name) {
    int i;
    int pos = SearchDatabase(personDatabase, Name);

    if (pos == -1) {
        printf("Person Not Found!\n");
        return;
    }

    FreePerson(personDatabase->Data[pos]);

    for (i = pos; i < personDatabase->peopleCount - 1; i++) {
        personDatabase->Data[i] = personDatabase->Data[i + 1];
    }
    personDatabase->peopleCount--;
}
```

Ders sonrası not: Yukarıdaki kod düzeltilmiş hâldir. Kısa nota göre ekrandaki ilk sürümde büyük olasılıkla `return` yoktu. O durumda kişi bulunamayınca `Data[-1]` serbest bırakılmaya çalışılır. Bu sürüm AddressSanitizer ile çalıştırıldı ve `heap-buffer-overflow` hatası verdi. Yukarıdaki sürümde bu hata çıkmadı.

Ders sonrası not: Kısa nota göre ilk sürümdeki döngü sınırı büyük olasılıkla `i < peopleCount` idi. Son adımda `Data[peopleCount]` okunur ve bu kayıtlı bir eleman değildir. Sınır `i < peopleCount - 1` olmalıdır. `AddPerson` dizi dolar dolmaz büyüttüğü için bu göz ayrılmış alanın içinde kalır, bu yüzden AddressSanitizer bu sürümde hata vermedi; yanlışlık yine de düzeltilmelidir.

8.3 Son Elemanı Silinen Yere Taşımak

Hoca kaydırmaya alternatif bir yöntem de söyledi. Kayıtlar sıralı değilse, silinen yeri doldurmak için bütün elemanları kaydırmaya gerek yoktur. En sondaki kişi alınıp silinen kişinin yerine konabilir. Böylece her şeyi kaydırmak gerekmez. Bu yöntemde elemanların sırası değişir, bu nedenle ancak sıranın önemli olmadığı durumda kullanılır.

9. Bütün Belleği Serbest Bırakma: FreeDatabase

Menüde `0` seçildiğinde program çıkmadan önce ayrılan bütün belleği serbest bırakmalıdır. Hoca bunun için kişiler tek tek gezilebileceğini söyledi. Kısa nottaki sürümde, kayıt kaldığı sürece sondaki kişi `FreePerson` ile serbest bırakılır ve `peopleCount` bir azaltılır. Kişiler bitince dizinin kendisi de dinamik olduğu için serbest bırakılır.

```
void FreeDatabase(DATABASE *personDatabase) {
    while (personDatabase->peopleCount > 0) {
```

```
        FreePerson(personDatabase->Data[personDatabase->peopleCount - 1]);
        personDatabase->peopleCount--;
    }
    free(personDatabase->Data);
}
```

Dizi tabanlı sürüm böylece tamamlanır.

10. Bağlı Liste Yapısına Geçiş

İkinci aşamada aynı program bağlı liste ile yazılır. Genel kod benzer olacağı için hoca tekrar etmedi; yalnızca değişecek fonksiyonların içine baktı. `JOB` ve `PERSON` yapıları aynen kalır. `initNewPerson`, `PrintPerson` ve `FreePerson` da değişmeden kullanılabilir.

Artık veritabanında kişiler doğrudan değil, düğüm (node) içinde tutulur. Düğüm iki şey içerir: bir kişi ve aynı tipten bir işaretçi. Bağlı liste yapmak için bu işaretçi gereklidir, çünkü her düğüm kendisinden sonraki düğümü gösterir. Son düğümün `Next` alanı `NULL`'dır. `Next` sayesinde bütün liste dolaşılabilir.

```
typedef struct node {
    PERSON *Person;
    struct node *Next;
} NODE;
```

Veritabanı için ayrı bir struct gerekmez. Listenin başını gösteren `NODE *head` işaretçisi yeterlidir ve başlangıçta `NULL` yapılır.

11. Listenin Başına Ekleme: AddPersonFirst

Hoca eklemenin iki yolunu ayırdı. Liste sıralı değilse en başa eklemek yeterlidir: yeni düğüm oluşturulur, `head` artık bu düğümü gösterir, yeni düğümün `Next`'i ise daha önce `head`'in gösterdiği yeri gösterir. En sona eklemek istersek `NULL` görene kadar gidip düğümü sona bağlarız. Önce başa ekleme yazıldı.

Fonksiyonda yeni bir düğüm oluşturulur ve kişisi atanır. Asıl işlem iki adımdır:

1. Yeni düğümün `Next` değeri, `head`'in eski değeri olur.
2. `head`'in yeni değeri, yeni düğümün adresi olur.

`head` fonksiyon içinde değiştiği için fonksiyona işaretçinin kendisi değil adresi gönderilir. Yani `head` zaten bir `NODE *` olduğundan parametre `NODE **` olur.

```
void AddPersonFirst(NODE **head, PERSON *newPerson) {
    NODE *newNode = (NODE *) malloc(sizeof(NODE));
    newNode->Person = newPerson;
    newNode->Next = *head;
    *head = newNode;
}
```

> **Hocanın vurgusu:** Baş işaretçisi değişebilecekse fonksiyona işaretçinin adresi (`NODE **`) gönderilir.

12. Listeyi Dolaşma ve İlk Deneme

Listelemede `head` değişmeyecektir. Bu nedenle adresini değil, kendisini geçmek yeterlidir. `head`'den başlanır ve `NULL` görene kadar `Next` ile ilerlenir. Bunun için `current` adlı bir işaretçi kullanılır. `current` `NULL` olmadığı müddetçe döngü devam eder. Her adımda `current`'ın kişisi `PrintPerson` ile yazdırılır, sonra `current = current->Next` yapılır.

```
void ListDatabase(NODE *head) {
    NODE *current = head;
    while (current != NULL) {
        PrintPerson(current->Person);
        current = current->Next;
```

```
    }
}
```

Kısa nota göre derste 1, 2, 3 eklenince liste 3, 2, 1 sırasıyla yazdı. Bunun nedeni her yeni kişinin en başa eklenmesidir. Liste son durumda şöyledir:

```
head -> [3] -> [2] -> [1] -> NULL
```

> **Ders sonrası not:** Bölüm 16 sonundaki birleştirilmiş kod bu girdiyle çalıştırıldı ve aynı sıra alındı.

13. Listenin Sonuna Ekleme: AddPersonLast

Sona eklemede düğüm yine aynı şekilde oluşturulur. Ancak bu kez yeni düğüm listenin sonuna gideceği için `Next`'i `NULL` olur.

Burada iki ihtimal vardır:

1. Hiç eleman yoksa, yani `*head == NULL` ise, yeni düğüm `head` yapılır.
2. Eleman varsa, listeleme koduna benzer şekilde sona kadar gidilir ve son düğümün `Next`'ine yeni düğüm atanır.

Hoca buradaki farkı özellikle belirtti. Listelemede `current` `NULL` olana kadar ilerliyorduk. Burada öyle yapılırsa elde `NULL` kalır ve `Next` alanı değiştirilemez. Bir son düğümün elimizde kalması gerektiği için `current->Next` `NULL` olana kadar ilerlenir. Döngü bitince `current` son düğümü gösterir ve `current->Next`'e yeni düğüm atanır.

```
void AddPersonLast(NODE **head, PERSON *newPerson) {
    NODE *current;
    NODE *newNode = (NODE *) malloc(sizeof(NODE));
    newNode->Person = newPerson;
    newNode->Next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    current = *head;
    while (current->Next != NULL) {
        current = current->Next;
    }
    current->Next = newNode;
}
```

14. Listede Arama

Aramada da `head` değişmeyeceği için kendisi gönderilir. Liste yine baştan `NULL` görene kadar dolaşılır. Her düğümde `current->Person->Name` ile aranan isim `strcmp` ile karşılaştırılır. Eşleşme varsa sonuç döndürülür, yoksa `current->Next`'e geçilir. Hiçbir şey bulunamazsa, dizideki gibi "bulunamadı" sonucu verilir.

```
int SearchDatabase(NODE *head, char *Name) {
    NODE *current = head;
    int index = 0;
    while (current != NULL) {
        if (strcmp(current->Person->Name, Name) == 0) {
            printf("Person Found!\n");
            return index;
        }
        current = current->Next;
        index++;
    }
```

```
    return -1;
}
```

15. Listeden Silme: DeleteDatabase

Silmede fonksiyonun başı aynıdır, içi biraz farklıdır. İlk düğüm silinirse `head` değişebileceği için yine `head`'in adresi gönderilir.

Burada arama, silme fonksiyonunun içinde yapılır. Çünkü yalnızca indisi bilmek yetmez. Hoca bunu bağlı listenin bir dezavantajı olarak anlattı: düğümün listede nerede olduğunu bilsek bile ona doğrudan erişemeyiz. Yine baştan oraya kadar gitmemiz gerekir, çünkü indis yoktur. Dizide olduğu gibi indisle tek seferde erişim mümkün değildir.

Silmek için `Next` ile ilerlerken önceki düğümü de bilmemiz gerekir. Silinecek düğüm bulunduğunda bir önceki düğümün `Next`'i, silinen düğümden sonraki düğümü göstermelidir. Bu yüzden `current`'a ek olarak `previous` adlı ikinci bir işaretçi tutulur. İlk düğüm silinecekse önceki düğüm yoktur; bu durumda `head`, ilk düğümün `Next`'ine eşitlenir ve geri kalan işlem aynıdır.

Adımlar şöyledir:

1. `previous = NULL`, `current = *head` ile başlanır.
2. `current` `NULL` değilken ve isim eşit değilken ilerlenir. Önce `previous = current`, sonra `current = current->Next` yapılır. İsim eşitse döngüden çıkılır.
3. Döngü sonunda `current` `NULL` ise listenin sonuna gelinmiş, kişi bulunamamıştır. Bulunamadı yazılır ve çıkılır.
4. `previous` `NULL` ise silinecek düğüm ilk düğümdür, `*head = current->Next` yapılır.
5. Değilse `previous->Next = current->Next` yapılır. Böylece aradaki düğüm zincirden çıkar.
6. Son olarak kişi `FreePerson` ile, düğümün kendisi de `free` ile serbest bırakılır.

```
void DeleteDatabase(NODE **head, char *Name) {
    NODE *previous = NULL;
    NODE *current = *head;

    while (current != NULL && strcmp(current->Person->Name, Name) != 0) {
        previous = current;
        current = current->Next;
    }

    if (current == NULL) {
        printf("Person Not Found!\n");
        return;
    }

    if (previous == NULL) {
        *head = current->Next;
    } else {
        previous->Next = current->Next;
    }

    FreePerson(current->Person);
    free(current);
}
```

> **Hocanın vurgusu:** Bağlı listeden silerken bir önceki düğüm (`previous`) takip edilir.
>
> **Ders sonrası not:** Düğüm de `malloc` ile oluşturulduğu için ayrıca `free(current)` çağrılır. Altyazıda 01:19:18'de düğümün kendisinin de serbest bırakılacağı söyleniyor. Bu satır çıkarılıp kod çalıştırıldığında LeakSanitizer bellek sızıntısı bildirdi.

16. Listeyi Serbest Bırakma ve main

Program kapanırken baştan sona her düğüm serbest bırakılır. Burada bir tuzak vardır: mevcut düğüm silinmeden önce sonraki düğümün adresi bir yere alınmalıdır. Hoca bunu şöyle

anlattı: önce silersek sonraki düğümün adresini kaybederiz. Bu yüzden önce `Next` saklanır, sonra mevcut düğüm silinir, sonra saklanan adrese geçilir.

```
void FreeDatabase(NODE *head) {
    NODE *current = head;
    NODE *next;
    while (current != NULL) {
        next = current->Next;
        FreePerson(current->Person);
        free(current);
        current = next;
    }
}
```

Ders sonrası not: Önce silip sonra `current->Next` okuyan sürüm denendi. `gcc -Wall -Wuse-after-free` uyarısı verdi, AddressSanitizer de çalışırken `heap-use-after-free` hatası verdi.

Ders sonrası not: Aşağıdaki `main` altyazı ve kısa nottan birleştirilmiştir. İki ekleme fonksiyonunu birlikte denemek için 5 seçeneği eklendi; bu menü, metinleri ve `scanf` satırları derstekinden büyük olasılıkla farklıdır. Bölüm 2'deki `JOB` ve `PERSON`, Bölüm 4'teki `initNewPerson`, Bölüm 6'daki `PrintPerson`, Bölüm 8'deki `FreePerson` ve Bölüm 10-16'daki kod tek dosyada `gcc -Wall` ile uyarısız derlendi. AddressSanitizer ile başa ekleme, sona ekleme, ortadan ve baştan silme, olmayan ismi silme ve çıkış denendi; bellek hatası çıkmadı.

```
int main(void) {
    NODE *head = NULL;
    int exitControl = 1, choice, Age;
    char Name[100], jobName[100];

    while (exitControl) {
        printf("1-AddFirst 2-List 3-Search 4-Delete 5-AddLast 0-Exit\n");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            printf("Name, Age, JobName: ");
            scanf("%99s %d %99s", Name, &Age, jobName);
            AddPersonFirst(&head, initNewPerson(Name, Age, jobName));
            break;
        case 5:
            printf("Name, Age, JobName: ");
            scanf("%99s %d %99s", Name, &Age, jobName);
            AddPersonLast(&head, initNewPerson(Name, Age, jobName));
            break;
        case 2:
            ListDatabase(head);
            break;
        case 3:
            printf("Name: ");
            scanf("%99s", Name);
            if (SearchDatabase(head, Name) == -1) {
                printf("Person Not Found!\n");
            }
            break;
        case 4:
            printf("Name: ");
            scanf("%99s", Name);
            DeleteDatabase(&head, Name);
            break;
        case 0:
```

```
            FreeDatabase(head);
            exitControl = 0;
            break;
        default:
            printf("Wrong choice!\n");
        }
    }
    return 0;
}
```

17. Dinamik Dizi ve Bağlı Liste Karşılaştırması

İki kod da yazıldıktan sonra hoca, dizi ile bağlı listenin avantaj ve dezavantajlarını karşılaştırdı.

- **Büyüme:** Bağlı listeye bellek el verdiği sürece eleman eklenebilir. Dizide `databaseCount`, `peopleCount` değerine ulaştığında boyutu iki katına çıkarmak ve yeniden yer ayırmak gerekir.
- **Silme:** Bağlı listede eleman silinirken yalnızca ilgili düğüm zincirden kaldırılır, kaydırma gerekmez. Dizide silinen yerin boş kalması istenmiyorsa ve elemanlar sıralıysa hepsi kaydırılır.
- **Bellek:** Bellek açısından bağlı liste daha fazla yer harcar. Dizide her kişi için yalnızca kişinin işaretçisi vardı. Düğüm kullanıldığında ayrıca `Next` işaretçisi de tutulur ve bu ek bellek demektir.
- **Erişim:** Dizide kişinin indisi bulunduğu anda doğrudan ona gidilebilir. Bağlı listede konum bilinse bile doğrudan erişilemez, baştan `next, next, next` diye gidilir. Bu yüzden elemana erişim $O(n)$ karmaşıklığındadır.

Hocanın sonucu şudur: çok ekleme ve çıkarma yapılan durumlarda bağlı liste daha doğru veri yapısıdır ve işlem bakımından çok daha az işlem yapar. Ama eleman sayısı kısıtlıysa dinamik dizi tercih edilebilir. Genel olarak bağlı liste daha dinamiktir.

> **Hocanın vurgusu:** Sık ekleme ve silme varsa bağlı liste, sınırlı sayıda eleman ve sık indisli erişim varsa dizi uygundur.
>
> **Ek bilgi (derste anlatılmadı):** "Daha az işlem" karşılaştırması büyük olasılıkla düğüm bulunduktan sonraki bağlantı değişikliği için geçerlidir. Derste yazılan `DeleteDatabase` önce listeyi baştan dolaşarak düğümü arar, `AddPersonLast` da sona kadar yürür. Bu iki işlemde dolaşma maliyeti kalkmaz; başa ekleme ise dolaşma gerektirmez.

18. Başlık Dosyaları ve Makrolar

Karşılaştırmadan sonra hoca son bir konu olarak başlık dosyalarını ve makroları gösterdi. Örnek program `header.c`, kendi yazdığımız `menu.h` ve `calculation.h` dosyalarını kullanır.

18.1 Açılı Parantez ve Tırnak ile include

Başlık dosyaları `#include` ile eklenir ve iki yazımı vardır:

- `#include <stdio.h>`: Açılı parantez kullanıldığında derleyicinin bulunduğu dizine bakılır.
- `#include "menu.h"`: Tırnak içinde yazıldığında mevcut dizine, yani bu dosyanın bulunduğu dizine bakılır ve oradaki dosya dahil edilir.

> **Ek bilgi (derste anlatılmadı):** Tırnaklı yazım dosyayı mevcut dizinde bulamazsa standart dizinlere de bakar. gcc 15.2 ile denendi: `#include "stdio.h"` derlendi. Tersi geçerli değildir: mevcut dizindeki bir başlık açılı parantezle yazılınca derleyici "No such file or directory" hatası verdi.

18.2 Include Koruması

`menu.h` dosyası `#ifndef MENU_H` ile başlar. Bu ifade "if not defined" demektir: `MENU_H` tanımlı değilse devam edilir ve hemen arkasından `#define MENU_H` ile tanımlanır. Dosyanın sonunda

`#endif` vardır. Bunun adı include korumasıdır. Amaç, aynı dosya birden fazla kez include edilirse tanımların tekrar etmesinden doğacak karışıklığı önlemektir.

> **Ders sonrası not:** Altyazıda 01:27:57'de "daha önce include edilirse onu include et" gibi bir cümle var, ama konuşma net çözülmemiş olabilir. Doğrusu şudur: dosya daha önce dahil edilmediyse içeriği işlenir, dahil edildiyse atlanır.
>
> **Ek bilgi (derste anlatılmadı):** Korumanın etkisi gcc 15.2 ile denendi. Başlıkta `int sayac = 1;` gibi bir tanım varken dosya korumasız iki kez dahil edilince "redefinition" hatası çıktı, korumayla derlendi. Aynı struct tanımının tekrarı ise gcc 15.2'nin varsayılan standardında hata vermedi, `-std=c17` ile hata verdi.

18.3 MENU Makrosu

`menu.h` içinde `MENU` adlı bir makro tanımlanır. Makrolarda, derleme yapılırken makronun adının yazıldığı yere tanımdaki ifade kopyalanır. Menüde birden fazla `printf` alt alta yazılır. Satır sonuna kaçış karakteri `\` konduğunda satırların birbirinin devamı olduğu belirtilir. `header.c` derlendiğinde `MENU` yazan yere bu ifadeler konur.

```
#ifndef MENU_H
#define MENU_H

#define MENU printf("1-Toplama\n"); \
             printf("2-Cikarma\n"); \
             printf("3-Carpma\n"); \
             printf("0-Cikis\n");

#endif
```

18.4 calculation.h ve header.c

`calculation.h` dosyasında benzer şekilde include koruması vardır. Toplama için `topla(x, y)` makrosu tanımlanır: derleyici `topla(x, y)` ifadesini gördüğü yere, derleme zamanından önce `x + y` ifadesini koyar. Yani yer değiştirme önişlemci aşamasında olur.

```
#ifndef CALCULATION_H
#define CALCULATION_H

#define topla(x, y) ((x) + (y))
#define cikar(x, y) ((x) - (y))
#define carp(x, y)  ((x) * (y))

#endif
```

> **Ders sonrası not:** Altyazı tanımı "x artı y" diye anlatıyor, kısa not ise `((x) + (y))` biçimini veriyor. Parantezli yazım gerekir. Kısa nottaki örnek gcc ile çalıştırıldı ve python ile hesaplandı: `#define topla(x, y) x + y` için `2 * topla(3, 4)` ifadesi $2 \cdot 3 + 4 = 10$ verir, parantezli tanımda $2 \cdot (3 + 4) = 14$ olur.

Bu sayede bütün ayrı dosyaları elle kopyalamak yerine tek bir satırla hepsi dahil edilebilir. Hocaya göre `stdio.h` gibi kütüphaneler de aslında benzer şekilde çalışır. Böylece programlar birbirinden bağımsız dosyalara ayrılmış olur. `header.c` bu iki dosyayı dahil ederek `switch` ile bir hesap makinesi menüsü kurar.

```
#include <stdio.h>
#include "menu.h"
#include "calculation.h"

int main(void) {
    int choice = 1, x, y;

    while (choice != 0) {
        MENU
```

```
        scanf("%d", &choice);

        if (choice >= 1 && choice <= 3) {
            scanf("%d %d", &x, &y);
        }

        switch (choice) {
        case 1:
            printf("%d\n", topla(x, y));
            break;
        case 2:
            printf("%d\n", cikar(x, y));
            break;
        case 3:
            printf("%d\n", carp(x, y));
            break;
        case 0:
            break;
        default:
            printf("Yanlis secim\n");
        }
    }
    return 0;
}
```

Ders sonrası not: Yukarıdaki iki başlık ve `header.c`, kısa nottaki tarife göre kuruldu. Menü metinleri, `cikar` ve `carp` makrolarının adları ve okuma satırları derstekinden büyük olasılıkla farklıdır. Üç dosya `gcc -Wall` ile uyarısız derlendi. 3 ve 4 girdisiyle toplama 7, çıkarma -1, çarpma 12 sonucunu verdi.

Dersin Toparlaması

Bu uygulamada aynı çalışan veritabanı iki veri yapısıyla yazıldı. Dizi sürümünde `PERSON **` ile işaretçi dizisi tutuldu, dizi dolunca `realloc` ile iki katına çıkarıldı, silmede elemanlar kaydırıldı. Bağlı liste sürümünde `NODE` ile zincir kuruldu, `head` değişebilen işlemlerde `NODE **` gönderildi, silmede `previous` takip edildi ve serbest bırakırken `Next` önceden saklandı. Her iki sürümde de `malloc` ile ayrılan her alan, önce iç alanlar olmak üzere serbest bırakıldı.

Karşılaştırmanın sonucu, seçimin işlem türüne bağlı olduğudur: sık ekleme ve silmede bağlı liste, sınırlı sayıda eleman ve indisli erişimde dinamik dizi öne çıkar. Son olarak başlık dosyaları, include koruması ve makrolarla programın bağımsız dosyalara nasıl ayrılacağı gösterildi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.